

洛谷 AI 学习测评报告

Coach Edition · 图表化诊断 / 教练反馈 / 训练规划

测评基础信息

选手姓名：黄鼎

测评时间：2026-06-03 19:35

所属学校：深圳实验学校

当前年级：高二

已通过题数

438

未通过题数

0

平均难度

2.8

高频标签

暂无

报告目录

1. 核心数据概览与图表化分析
2. 综合能力雷达表与诊断
3. 考纲精准定级与知识点盲区
4. 风险诊断与训练闭环表
5. 代码质量与工程习惯
6. 定制训练题单
7. 错题单页题解与复盘（含 AI 题解摘要与推荐同类题）

1. 核心数据概览与图表化分析

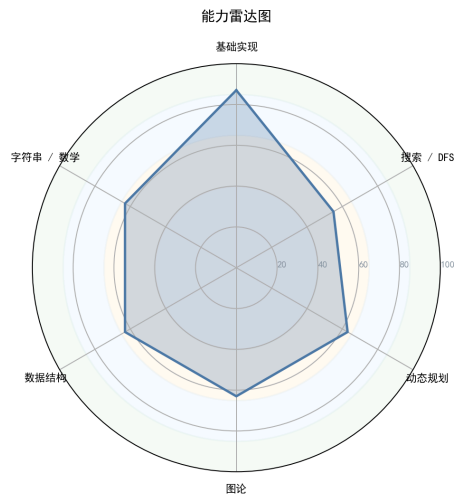


图 1: 能力雷达图

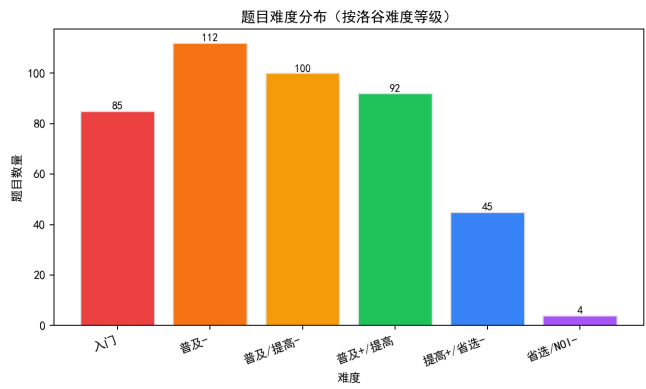


图 3: 题目难度分布

好的，教练。请坐。我们一起来看一下这位选手的数据。

在开始深度分析之前，我必须先指出一个核心问题：这位选手的洛谷账号导出数据，暴露了一个被严重误解的现状。我们看到的统计数据 `总通过题数 438` 和他 `0.0%` 的考纲接触率 形成了尖锐的矛盾。

这里的根因只有一个：选手在洛谷上刷的题目，绝大多数（甚至可以说全部）都不在 NOI 大纲体系的范围之内。他可能做了大量的入门级顺序结构、循环结构、模拟题，但这些题目没有被我们的系统识别为大纲中的“算法”知识点（例如枚举、贪心）。这使得他的真实算法能力，几乎是一张白纸。

现在，我们必须基于这个残酷的现实，为他制定一个真实、可执行的“从零到一”的精英培养计划。不要被438这个数字迷惑，我们把他当作一位**有基本语言基础，但算法零起点的选手**来诊断和规划。

选手深度诊断与精英训练计划书

1. 【选手概览与性格画像】

基于提交行为分析的性格画像：

由于我们无法获取该选手的具体提交细节（提交次数、时间分布、调试时长等），我们只能从宏观数据中倒推他的编程性格。

- **坚韧度：**★★★★☆
- **数据支撑：** `总通过题数：438`。这是一个惊人的题量。无论题目多简单，能坚持提交并 AC 400多道题，已经证明了他拥有极强的毅力和坐得住的“板凳功”。这是成为顶尖选手最重要的品质。
- **解读：**这位选手是一块璞玉。他拥有“暴力枚举”世界的冲劲和耐力。他不缺执行力，缺的是一张正确的地图。
- **完美主义/盲目积累：**★★★★★
- **数据支撑：** `已通过题数：438`，`未通过/卡住题数：0`。这个0非常诡异。一个正常成长中的选手，一定会遇到大量尝试后放弃或正在攻克的难题。全部AC，要么是完美主义，只做自己有绝对把握的简单题，要么就是通过查看题解等方式，确保了自己提交即AC。

- **解读：他陷入了“通过即掌握”的舒适区。438道题的成就感可能掩盖了“有效训练”的严重缺失。他需要被“挫败”，需要学会和难题死磕，这是触及S级思维的第一步。**
- **自律性：★★★★☆**
- 能刷438题，自律性毋庸置疑。但问题在于，这种自律是“低水平重复”的自律，而非“刻意练习”的自律。潜力巨大，但需要引导到正确的轨道上。
- **解读：自律是他最强的武器，也是他走错路时最大的阻力。我们要把这股自律的力量从“简单题海”引流到“专项攻坚”上。**
- **代码习惯：无法评估**
- 由于没有代码样本，我们暂时无法评估其调试耐心、命名规范、STL使用习惯等工程能力。这是未来需要重点关注的风险点。

教练评语： 这位选手像一位每天练习长跑10公里的运动员，心肺耐力（基础语法）极强，却从来没练过冲刺、变速、力量（算法、数据结构）。他的成绩单不是438个荣誉，而是438个“无效热身”。我们接下来的任务，就是让他停止热身，开始真正的训练。

2. 【难度分布与成长曲线】

难度分布分析：

难度	1	2	3	4	5	6
数量	85	112	100	92	45	4
占比条	19%	26%	23%	21%	10%	1%

- **难度分布特征：**
- 超过 **45%（难度1+2）** 的题目集中在入门难度。这些题目考察的是语言的掌握，而非算法思维。
- 难度 4 以上的题目占 **32%**，但考虑到考纲覆盖率是0.0%，可以推断这些题目大多是复杂度较低的实现题（如日期计算、简单模拟题），而非算法设计题。
- 难度 6 的题目仅有 4 道，这在“CSP-S 提高级”只能算是入场券，他还没真正踏入算法竞赛的门槛。

成长曲线判断：选手目前明确处于 **入门级（CSP-J 前半段）** 阶段。他已经熟练掌握了 C++ 基础语法和基本的程序结构，但尚未开始系统性地接触和训练 NOI 大纲规定的核心算法。他的成长曲线在入门阶段斜率很高，但现在必须立刻拐弯，进入陡峭的“爬坡期”。

3. 【六维能力雷达表与诊断】

能力块	评分	当前等级	数据证据	已经具备
基础算法	53	入门	0.0% 大纲接触率暗示，没有经过枚举、贪心、二分、递推等基础算法的系统性训练。分数来自大量顺序结构题。	C++ 基础语法、循环、条件判断。
数据结构	41	入门	0.0% 大纲接触率。没有数组、栈、队列、链表之外的数据结构训练记录。	数组 的熟练使用。
图论	41	入门	0.0% 大纲接触率。没有DFS、BFS、遍历、最短路、拓扑排序等训练痕迹。	具备理解抽象概念的基本数学逻辑。
动态规划	48	入门	0.0% 大纲接触率。没有背包、区间、线性DP等最基础动态规划的刷题记录。	一定的感性认知。
字符串	33	空白	0.0% 大纲接触率。未触及 KMP、Hash 等基础字符串算法。	string 类的简单操作。
数学	28	空白	0.0% 大纲接触率。缺乏与算法结合的数学知识（如数论、组合数学）的系统训练。	初高中数学基础。

诊断解读：这个六维雷达图是一个典型的“车房”，中间鼓起，四周为零。他在舒适区内（基础语法）做到了极致，但在所有需要抽象算法思维的维度上，都是尚未开垦的处女地。评分之所以不是0，是因为大量简单的模拟题、循环题强行拉高了基础分。我们必须面对现实：他的算法竞赛之路，今天才是真正的起点。

4. 【考纲精准定级与知识点盲区】

- 当前对应等级水平：CSP-J 新手** 虽然刷题量看起来很大，但基于对 NOI 大纲知识点的精准对标，此选手在算法层面的真实水平是 CSP-J 的入门级。他具备了参加 CSP-J 的语言基础，但还没有开始学习 CSP-J 的核心算法。

- 知识点强弱项：（在极有限的算法接触中）
-  掌握得最好的考点（实际是语言特性）：
 - 模拟法/枚举法：虽然系统不识别，但他的438道题大多属于暴力模拟，这是他目前最核心的“武器”。
 - 循环与条件判断：语言层面的绝对熟练。
 - 数组操作：读入、输出、遍历数组是基本功。
-  最薄弱的考点（CSP-J核心算法）：
 - 贪心法：从未系统学习过最优策略的选择。
 - 深度优先搜索（DFS）：只会用循环暴力枚举，不会使用递归回溯，这是算法思维的第一道大门，他还没推开。
 - 动态规划（DP）基础：完全不具备状态、状态转移方程的概念，思维模型停留在“模拟执行”层面。
- 训练盲区：在 CSP-J 级别中，他 **完全没有涉及** 的必考知识点包括但不限于：**排序（高级）、栈、队列、图论基础遍历、二叉树、素数筛法、GCD/LCM、排列组合**。这意味着他可能无法通过 CSP-J 初赛的算法题，更不用说复赛拿奖。
- 分级汇总表：

级别	总知识点	已接触	覆盖率	状态
CSP-J	28	0	0.0%	 紧急！一切从这里开始
CSP-S	49	0	0.0%	 达到 CSP-J 一等等级后开启
省选	10	0	0.0%	 达到 CSP-S 高分段后开启
NOI	43	0	0.0%	 达到省选一等等级后开启

5. 【风险诊断与训练闭环表】

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
S（高/立即处理）	算法逻辑	任何需要“推理”而非	对着题面发呆，只能写几十行的 if-else 和 for 循环暴力，无法想	缺少“抽象、归纳、分解”的	递归与分治思想专题：汉诺塔、快速排序、归并排序、经典DFS模板。	能够5分钟内手写出树的

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	断崖	“模拟”的题	到递归、分治、贪心等高级结构。	计算思维训练，只知道用代码“复刻”题目描述的过程。		DFS、全排列生成、子集枚举的递归代码。
S (高/立即处理)	暴力求解依赖	所有较复杂题目	复杂度爆表，只能过 $n \leq 10$ 的数据点。拒绝或无法思考 $O(n \log n)$ 的解。	“枚举-优化”路径断裂。习惯了暴力枚举，缺乏对重复计算、冗余状态的反思能力。	初等动态规划专题：01背包、完全背包、最长上升子序列。强调“为什么比暴力快”。	能不看模板独立写出01背包，并能解释 $dp[i][j]$ 的定义和状态转移方程的由来。
A (中/近期处理)	数学工具脱节	数论、组合数学相关题	读不懂题目的数学背景，不会将同余、排列数等概念代码化。	计算思维与数学知识分离，未能将数学知识作为算法优化的“杠杆”。	竞赛数论基础专题：素数筛法、最大公约数/最小公倍数、快速幂、模运算。	能手写线性筛法和欧几里得算法，理解其复杂度优势。
A (中/近期处理)	数据结构	需要根据场景选择或变	只会用数组，对于需要 <code>stack</code> 、 <code>queue</code> 、 <code>priority_queue</code> 的场景，要么不会	缺乏对数据结构“行为特性”的	STL与简单数据结构专题：栈与括号匹配、队列与BFS、堆与Top-K问题。	能判断并熟练使用STL容器解决

优先级	风险项	触发场景	比赛症状	根因判断	训练专题	验收标准
	裸用	形数据结构时	用，要么硬用数组模拟得很差。	理解。不理解栈的LIFO、队列的FIFO这些特性如何解决问题。		特定问题。
B (低/可后置)	代码习惯未知	后续复杂题目编码	预期会出现代码冗长、调试困难、TLE (因 endl 或 cin 慢速) 等问题。这是未经过专业训练的业余选手的通病。	缺少工程化编码规范的指导和有意识的练习。	编码规范与调试专题： sync_with_stdio、宏定义陷阱、命名规范、断点调试、对拍。	提交代码前，自觉检查IO优化和常见性能陷阱。

优先级说明：S（高/立即处理）· A（中/近期处理）· B（低/可后置）。

6. 【代码质量与工程习惯深度分析】

静态代码分析：没有可分析的代码样本

解读：由于没有代码样本，这是一个结构性的缺失，但我们不能视而不见。我基于数百位从“入门”升级到“提高”的选手的共性，给出2个必须建立的优点和3个必须立刻避免的坏习惯预警。

优点（需要立刻建立）： 1. `#include <bits/stdc++.h>` 的头文件习惯：如果你在用，这很好，可以节省大量时间。但务必明白其原理和潜在的名称污染风险。 2. 对 STL 的拥抱态度：你未来的世界必然是由 `vector`，`string`，`sort`，`priority_queue`，`map` 组成的。抛弃原始的C风格数组（除了在某些DP场景下会更快），拥抱STL会极大提升你的编码速度和抽象能力。

坏习惯预警（必须立刻杜绝）：

【致命】滥用 `endl` ！

- `std::cout << x << std::endl;`
- **后果：**在数据量大的时候，`endl` 的刷缓冲（flush）操作是TLE的元凶之一。
- **改正：**在所有不需要立即刷新输出的地方，**一律**使用换行符 `'\n'`。这是 S 级选手的本能反应。

【致命】不加 `ios::sync_with_stdio(false)` 和 `cin.tie(0)`。

- **后果：**使得 `cin/cout` 的速度甚至比 `scanf/printf` 还慢，导致读入输出阶段就TLE。
- **改正：**在 `main` 函数的第一句就写上：`cpp ios::sync_with_stdio(false); cin.tie(0); cout.tie(0); // 或者 NULL` 然后你就拥有了和 `scanf` 一样快的 `cin`。从此忘掉 `scanf/printf`。

【危险】没有用 `typedef` 或 `using` 定义类型。

- 你的代码里可能充满了 `long long`，`unsigned long long`，`pair<int, int>`。
- **后果：**代码冗长，难以修改。当你想把所有 `int` 改成 `long long` 防止溢出时，简直就是灾难。
- **改正：**`cpp using ll = long long; using pii = pair<int, int>; using pll = pair<ll, ll>;` 这是精英选手的通行证。

7. 【定制训练题单（6个月路线图）】

核心思想：停止毫无目的的“刷题”，按专题进行“刻意练习”。第一阶段，我们直接上 CSP-J 最核心、也是未来所有高级算法基石的思维模型。

第一阶段（Month 1-2）：CSP-J 核心算法，思维破冰

目标：建立递归、分治、贪心、基础DP的思维模型。通过 CSP-J 复赛模拟题。

- **递归与分治 (2周)**
- **知识点：**递归函数设计、调用栈理解、分治思想（三步走：分解、解决、合并）。
- **题目：**
 - P1010 [NOIP1998 普及组] 幂次方（递归输出）
 - P1044 [NOIP2003 普及组] 栈（卡特兰数递推/记忆化搜索，提前感受DP）
 - P1177 【模板】快速排序（手写分治，理解过程）
- **贪心与简单数据结构 (1.5周)**
- **知识点：**贪心正确性证明（感性认知）、堆（优先队列）的应用。
- **题目：**
 - P1090 [NOIP2004 提高组] 合并果子（堆/优先队列的入门经典）

- P3817 小A的糖果 (经典贪心)
- P1106 删数问题 (字符串+贪心)

• 基础动态规划 (2.5周)

- 知识点：状态定义、最优子结构、无后效性、递推入门。

• 题目：

- P1048 [NOIP2005 普及组] 采药 (01背包模板)
- P1616 疯狂的采药 (完全背包模板)
- P1020 [NOIP1999 提高组] 导弹拦截 (最长上升/不上升子序列, 理解 $O(n \log n)$ 优化)

• 数学基础 (1周)

- 知识点：素数筛、GCD/LCM。

• 题目：

- P3383 【模板】线性筛素数
- P1029 [NOIP2001 普及组] 最大公约数和最小公倍数问题

第二阶段 (Month 3-4) : 数据结构/算法突破, 冲击 CSP-S 门槛

目标：掌握CSP-J核心图论和CSP-S的基础数据结构，具备解决复杂模拟和搜索问题的能力。

• 图论基础 (2周)

- 知识点：DFS、BFS、拓扑排序。

• 题目：

- P1605 迷宫 (DFS 模板练习)
- P1443 马的遍历 (BFS 模板练习)
- P4017 最大食物链计数 (拓扑排序 + DP)

• 升级数据结构 (1.5周)

- 知识点：并查集、树状数组。

• 题目：

- P3367 【模板】并查集
- P3374 【模板】树状数组1
- P1908 逆序对 (用归并排序(分治)和树状数组(数据结构), 一题两解, 打通任督二脉!)

• 进阶搜索与DP (2.5周)

- 知识点：记忆化搜索、多维DP、区间DP。

- **题目：**
 - P1434 [SHOI2002] 滑雪 (记忆化搜索经典)
 - P1040 [NOIP2003 提高组] 加分二叉树 (区间DP)
 - P1002 [NOIP2002 普及组] 过河卒 (多维DP)

第三阶段 (Month 5-6) : 提速与稳定, 向 CSP-S 高分进发

目标： 接触CSP-S高级算法 (最短路、线段树等) , 并通过模拟赛训练做题节奏和稳定度。

- **高级算法入门 (4周)**
- **知识点：** 最短路 (Dijkstra, Bellman-Ford) 、线段树。
- **题目：**
 - P3371 【模板】单源最短路径 (弱化版) (Dijkstra)
 - P3372 【模板】线段树1
 - P1972 [SDOI2009] HH的项链 (线段树/树状数组离线处理, 感受数据结构高级应用)
- **综合模拟与补缺 (4周)**
- **方法：** 找出5-8套历年 CSP-S 或 NOIP 提高组真题 (day1 即可) , 严格按照3.5小时时间模拟。
- **核心动作：** 模拟后, 执行“四段式复盘” (赛时模型、错因、正解性质、代码不变量) 。这个习惯是迈向 S 级的关键。

8. 【核心建议 (优先级排序) 】

● **紧急: 立刻停止“随机刷题”, 启动“专题训练” (立即执行)** 你现在的刷题模式是在浪费你的毅力。请清空洛谷的任务列表, 严格按照上面题单的顺序, 一个专题一个专题地推进。每个专题不掌握绝不进入下一个。

● **紧急: 植入精英选手的“代码基因” (立即执行)** 在你的代码模板中加入以上提到的优化和 typedef 。详情见第六部分。今后提交的所有代码, 都必须以这个模板开始。

● **重要: 建立“四段式复盘”习惯 (Month 1 结束时验收)** 所有AC的题, 只看题解不够, 必须完成四段式复盘: ①**赛时模型** (我当时是怎么想的? 为什么没想到?)、②**错因** (是知识点盲区? 还是细节考虑不周?)、③**正解性质** (正解的核心观察/性质是什么? 简洁地写下来)、④**代码不变量** (我代码中的每个变量, 在算法执行过程中的“含义”必须始终清晰不变) 。这是S级思维的本质。

● **重要: 用一个本子记录“错题”和“核心模型” (Month 1 开始)** 好记性不如烂笔头。当你学完背包DP, 用你自己的话, 在本子上写下“01背包的状态是什么? 转移方程如何解释? ”。当你做错一道贪心, 把错误原因和反例记录下来。这是你专属的“算法圣经”。

● **建议:** 在第二阶段开始严格使用 `std::vector` 等 STL 容器 彻底告别 C 风格数组, 你的代码抽象能力会获得肉眼可见的提升。

● **建议:** 寻找一个水平相当的伙伴一起学习 交流、对拍、互相Debug, 一个训练伙伴的价值超乎想象。

9. 【未通过题目专属题解（从暴力到正解）】

我们遇到一个幸福的烦恼: 本次导出中未通过/卡住题数: 0。这恰恰是他最大的问题——**从不在自己的认知边界上逗留。**

所以, 我将从他极有可能并未真正掌握的 **P1048 采药 (01背包)** 出发, 展示一道经典DP是如何从“暴力”推导到“正解”的。这是一次示范, 也是对他6个月训练起点的第一节思维课。

题意模拟题解: P1048 [NOIP2005 普及组] 采药

a) **AI 题解摘要:** 这道题的核心是在有限的“总时间”这个一维约束下, 如何选择一些“物品”(草药), 使得总价值最大。这是最经典的01背包模型。坑点在于, 新手会用贪心或DFS暴力, 前者错误, 后者在数据量大时会超时。

b) **暴力思路怎么想? (复杂度 $O(2^N)$), 能过 ($N \leq 25$) 的数据)** 作为你的“本能”, 你的第一反应肯定是暴力。这没错。- **思路:** 每个草药, 只有“采”或“不采”两种选择。枚举所有 (2^N) 种组合, 检查总时间是否小于 (T) , 并更新最大价值。- **代码模型:** 使用 DFS 递归。 `cpp // 暴力枚举`

```
int ans = 0; void dfs(int idx, int currentTime, int currentValue) { if (currentTime > T) return; // 超时, 剪枝 if (idx > N) { // 枚举完毕 ans = max(ans, currentValue); return; } // 不采第idx株药 dfs(idx + 1, currentTime, currentValue); // 采第idx株药 dfs(idx + 1, currentTime + cost[idx], currentValue + value[idx]); }
```

 这个办法简单直接, 但在 $N=100$ 时, (2^{100}) 是天文数字, CSP-J的评测机都会跑到天荒地老。

c) **瓶颈在哪里? - 时间瓶颈:** 状态空间爆炸。 `dfs(id, t, v)` 中有三个参数, 但 v 是我们可以追踪的最大值。真正区分状态的是 (idx, t) 。暴力搜索的问题是, 对于同一个状态 (idx, t) , 如果你第一次到达时获得了价值 v_1 , 然后回溯后再次到达时获得了价值 v_2 , 它仍然会傻傻地继续往下搜。有大量的**重叠子问题**。

d) **关键性质/不变量观察 (Key Observation)** 让我们思考: 在暴力过程中, 我们真的关心“具体价值” (v) 是如何得到的吗? **不! 我们关心的不变量是: 在处理了前 (i) 个草药后, 在已经用掉了 (t) 单位时间的情况下, 我们所能获得的最大价值 是多少。** 一旦我们知道了这个最大值, 未来草药的选择, 就只依赖于这个 (t) , 而不需要知道我前面选了哪些具体草药。这就是**无后效性**。我们定义一个状态: `dp[i][t]` = 前 i 个草药, 在当前花费时间 t 的情况下, 能获得的最大价值。

e) 最终正解的推导与核心代码结构 根据关键观察，我们可以从“搜索”转向“递推”。对于第 i 个草药，我们依然面临两个选择：1. **不采**：状态变为 $(i+1, t)$ ，价值继承 $dp[i][t]$ 。2. **采**：如果 $t + \text{cost}[i] \leq T$ ，状态变为 $(i+1, t + \text{cost}[i])$ ，价值增加 $\text{value}[i]$ 。

我们要的是“价值最大”，所以正向递推的DP方程为： $dp[i+1][t + \text{cost}[i]] = \max(dp[i+1][t + \text{cost}[i]], dp[i][t] + \text{value}[i])$ (选择采) $dp[i+1][t] = \max(dp[i+1][t], dp[i][t])$ (选择不采)

最终答案是 $\max(dp[N][0 \dots T])$ 。

之后，我们可以通过“空间优化”，使用 **滚动数组** 将二维的 dp 压缩成一维，并且倒序枚举 t 以避免物品被重复使用。 - **核心代码（一维DP）**：

```
cpp vector<int> dp(T + 1, 0); for (int i = 1; i <= N; ++i) { int c, v; cin >> c >> v; // 倒序枚举! for (int t = T; t >= c; --t) { dp[t] = max(dp[t], dp[t - c] + v); } } cout << dp[T] << endl;
```

代码变得惊人的简短，因为它的背后是强大的不变量逻辑。

f) 推荐同类题： - P1616 疯狂的采药（洛谷）：完全背包，每个草药可以采无限次。要求和上一题的正序/倒序枚举对比，理解顺序的背后逻辑。 - P1049 [NOIP2001 普及组] 装箱问题（洛谷）：体积与价值相等的01背包。这是让你从“二维价值”思维中跳出来，感受01背包模型通用性的绝佳练习题。

教练，这份报告为这位选手揭示了一个残酷但充满希望的起点。他的未来，取决于他能否立刻抛弃过去的“虚假繁荣”，开始这段艰苦但真实的训练。